

45-MINUTE DEEP DIVE

UMBRACO
INDIA FESTIVAL 26

I Was Building AI Agents the Wrong Way.

Here's What MCP Changed.

Architecture • Live demo • Security • Production lessons

Stephen Simon

Microsoft MVP (AI) • Global AI Community

Stephen SIMON

Program Manager - Global AI Community
Microsoft MVP in AI



My agent worked.

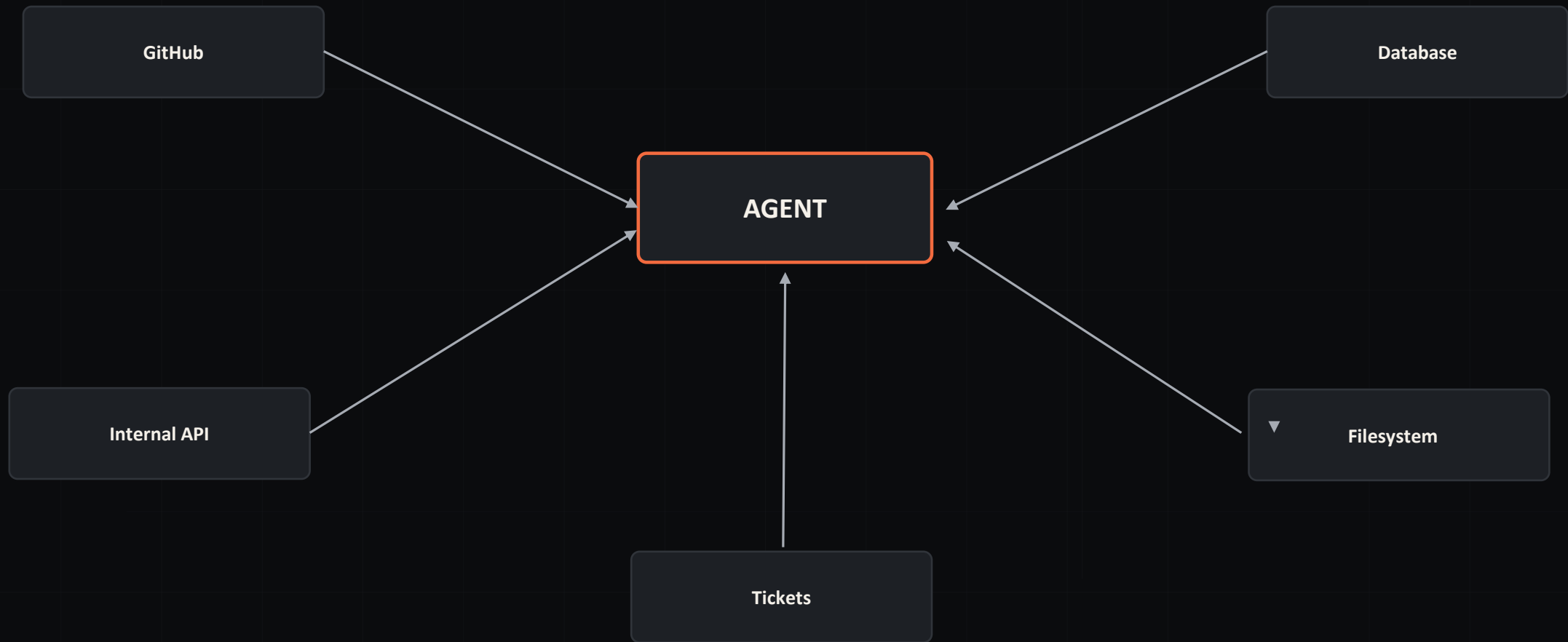
- ✓ It answered questions
- ✓ It could call tools
- ✓ It impressed people in the demo

The dangerous assumption

“If the model can reach a system,
the integration problem is solved.”

Until I gave it tools.

Every capability adds contracts, credentials, schemas, retries, errors and permissions.



“Just one more integration.”

WEEK 1

GitHub

Useful

WEEK 2

Database

Still fine

WEEK 3

Jira

Okay...

WEEK 4

Internal API

More glue

WEEK 6

Who owns all this?

Architectural debt

Mistake #1 — My agent knew too much.



agent.py

```
github_client()  
database_client()  
jira_client()  
auth()  
retry()  
schemas()  
error_handling()
```

The agent became...

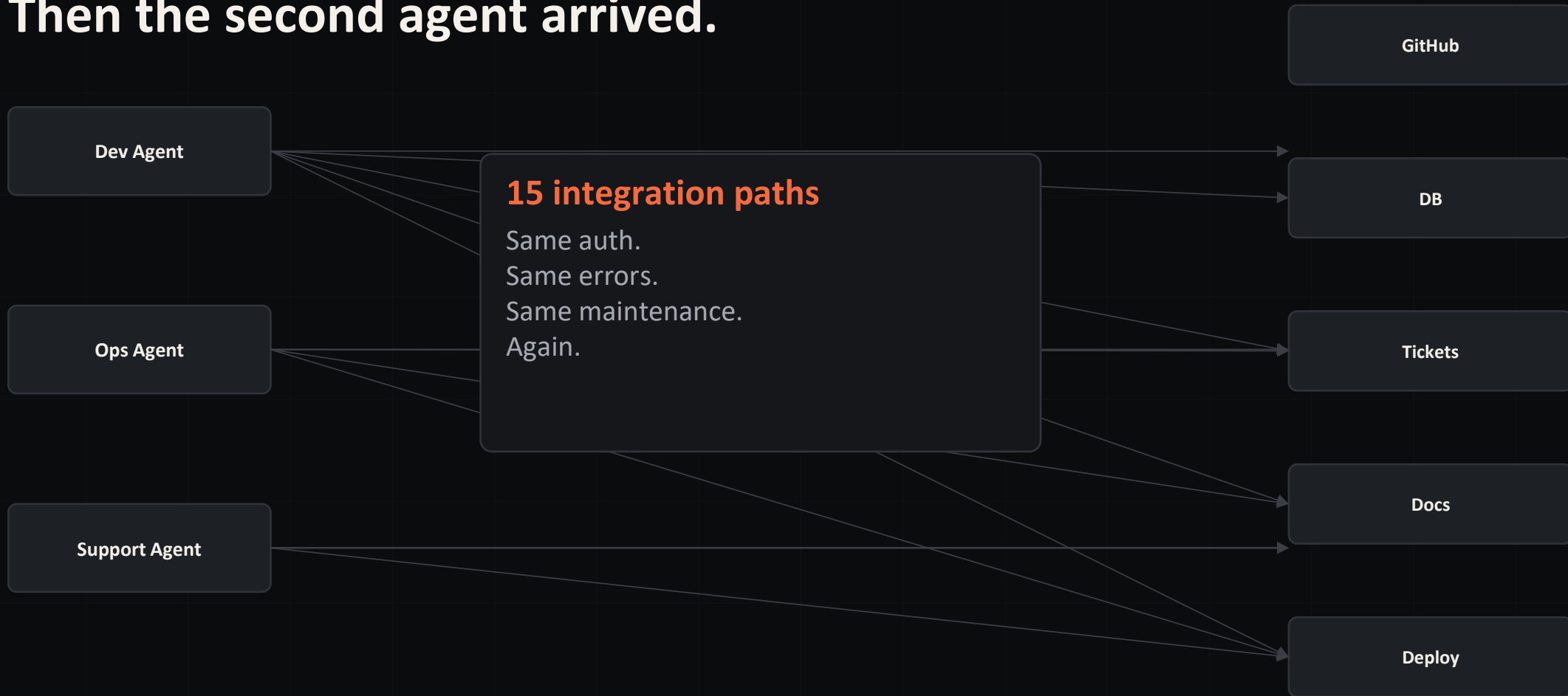
the brain

+

the plumbing

Downstream auth change?
API shape change?
Another AI app? → agent code changes.

Then the second agent arrived.



Mistake #2 — Tool calling ≠ integration architecture.

TOOL CALLING

How can the model ask my application to run a function?

MODEL ↔ APP

MCP

How can AI applications discover and interact with capabilities through a standard protocol?

APP ↔ CAPABILITY

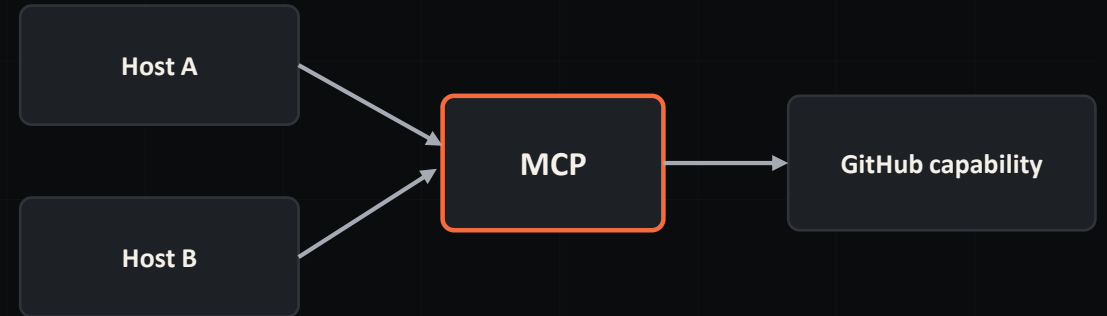
MCP does not replace tool calling. It standardizes the layer around capability exposure.

Mistake #3 — My integrations were not portable.

BEFORE



AFTER



The unit of reuse becomes a capability, not a code snippet inside one agent.

Then I found

MCP.

**A contract between AI applications
and the capabilities they need.**

The three actors that matter.

HOST

The AI application users interact with.

Coordinates the experience and manages one or more MCP clients.

MCP CLIENT

The protocol-level connection from the host to one MCP server.

Handles discovery and requests.

MCP SERVER

A program that exposes capabilities to AI applications.

Can be local or remote.

Host ≠ client ≠ server

Tools. Resources. Prompts.

TOOLS

Do something

create_issue()
deploy_service()

RESOURCES

Know something

runbook://payments
schema://orders

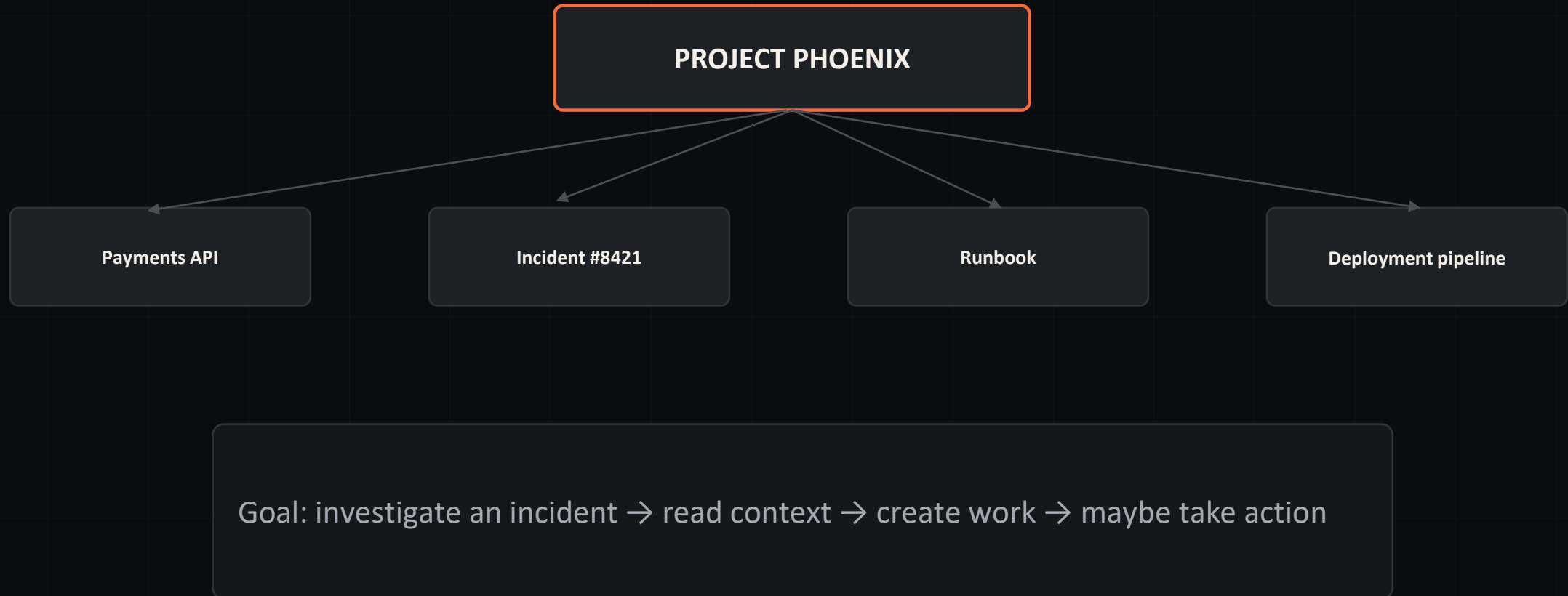
PROMPTS

Know how to approach something

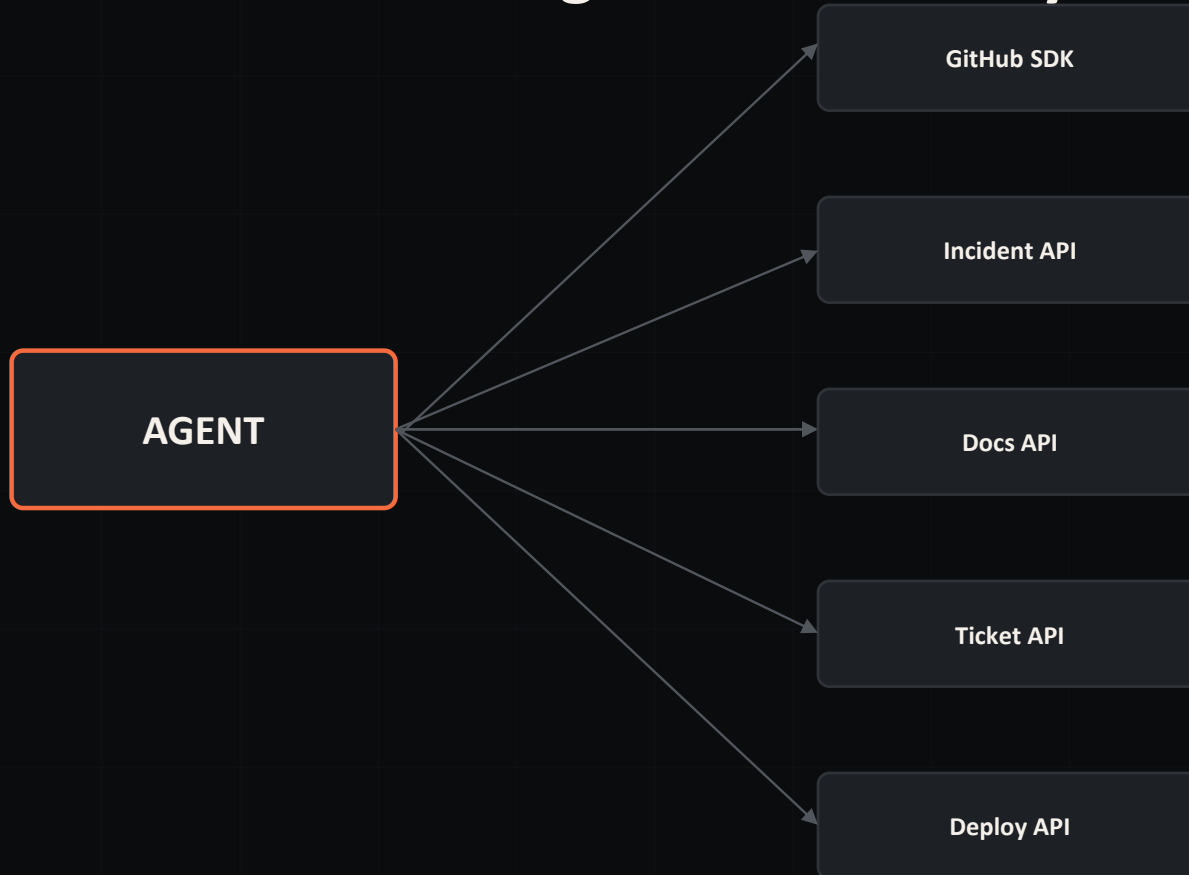
triage_incident
review_release

Tools act. Resources inform. Prompts guide.

Meet Acme Engineering.



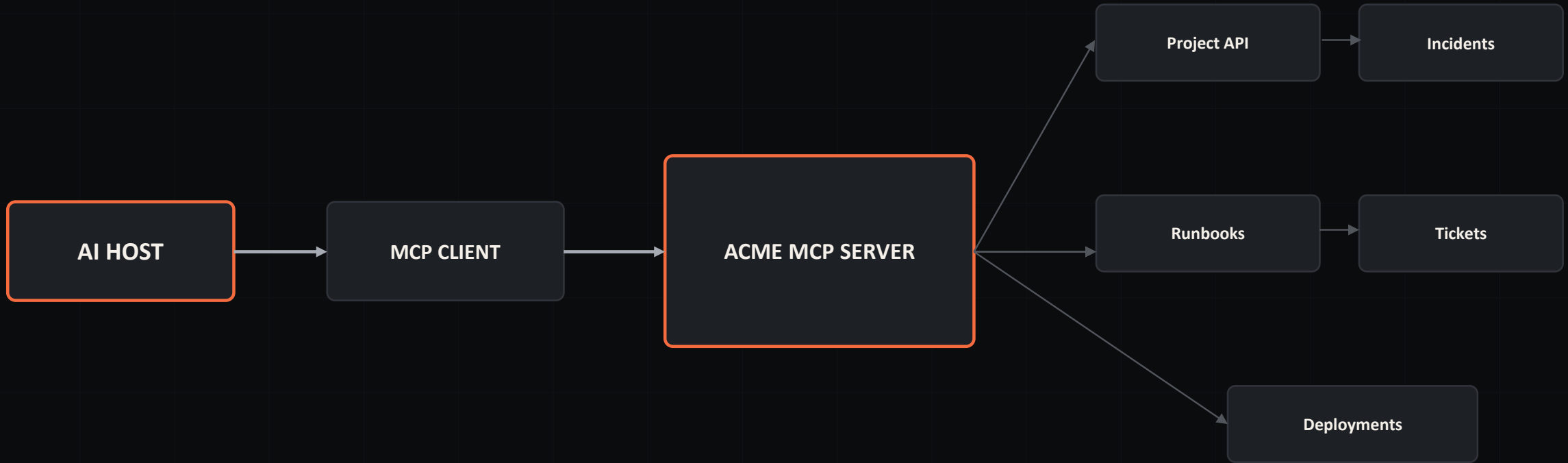
Before MCP: the agent owns every wire.



Inside the agent app

- Downstream SDKs
- Credentials
- Schemas
- Retries
- Error handling
- Tool semantics

After MCP: capabilities get their own boundary.

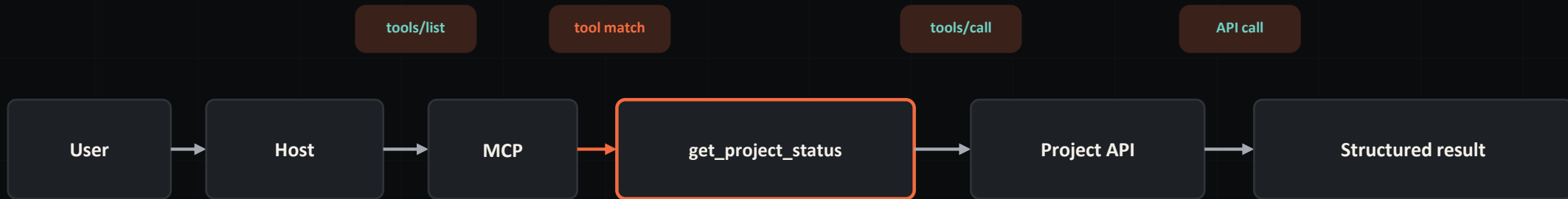


MCP does not replace your APIs. It defines the AI-facing capability boundary.

Demo 1 — Give the agent one useful capability.

01

“What is the status of Project Phoenix?”



Watch the boundary: discovery → model choice → routed call → structured result

What MCP changed #1

Integration became
infrastructure.

A capability can be designed, operated and reused independently of one agent.

Demo 2 — Add a write capability without rewriting the agent.

BEFORE

```
get_project_status(project)
```

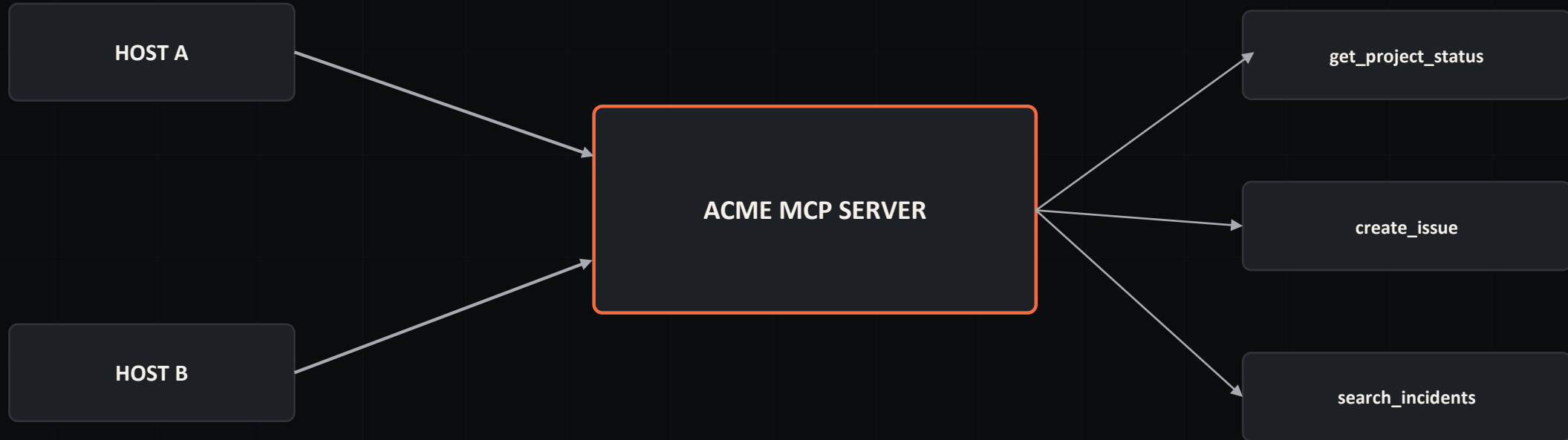


AFTER

```
get_project_status(project)  
  
+ create_issue(title, severity, service)
```

Same host. Same model. New capability.

Build the capability once. Use it from another host.



The host can change. The capability contract stays.

Then I made another mistake...

**I gave the agent
too much power.**

Capability ≠ Permission

AUTHENTICATION

Who are you?

AUTHORIZATION

What may you do?

TOOL AVAILABILITY

What should the model even see?

USER APPROVAL

Should this action happen now?

“The model has the tool” is not a security model.

MCP does not magically secure your agent.

Secret exposure

Tokens / credentials

Scope creep

Too much privilege

Tool poisoning

Malicious tool output

Command injection

Untrusted input → execution

Weak authorization

Identity / policy gaps

Missing audit trail

No forensic history

Shadow MCP servers

Unmanaged deployments

Context over-sharing

Data crosses boundaries

Design narrow tools, not superpowers.

BAD

```
execute_arbitrary_http_request(url, method, body)
```

BETTER

```
create_support_ticket(title, severity, service)
```

HIGH-RISK ACTION

```
deploy_service(service, approved_version, change_id) + server-side policy checks
```

Do not outsource policy to the prompt.

Autonomous does not have to mean unsupervised.

AGENT WANTS TO EXECUTE

deploy_service

service: payments

version: 2.4.1

change_id: CHG-1842

Cancel

Approve

MCP in 2026 looks more like infrastructure.

01

Stateless protocol core

Any request can land on any instance

02

Mcp-Method / Mcp-Name

Gateway-friendly routing & policy

03

Cacheable lists

Tool/resource catalogs can carry cache hints

04

Authorization hardening

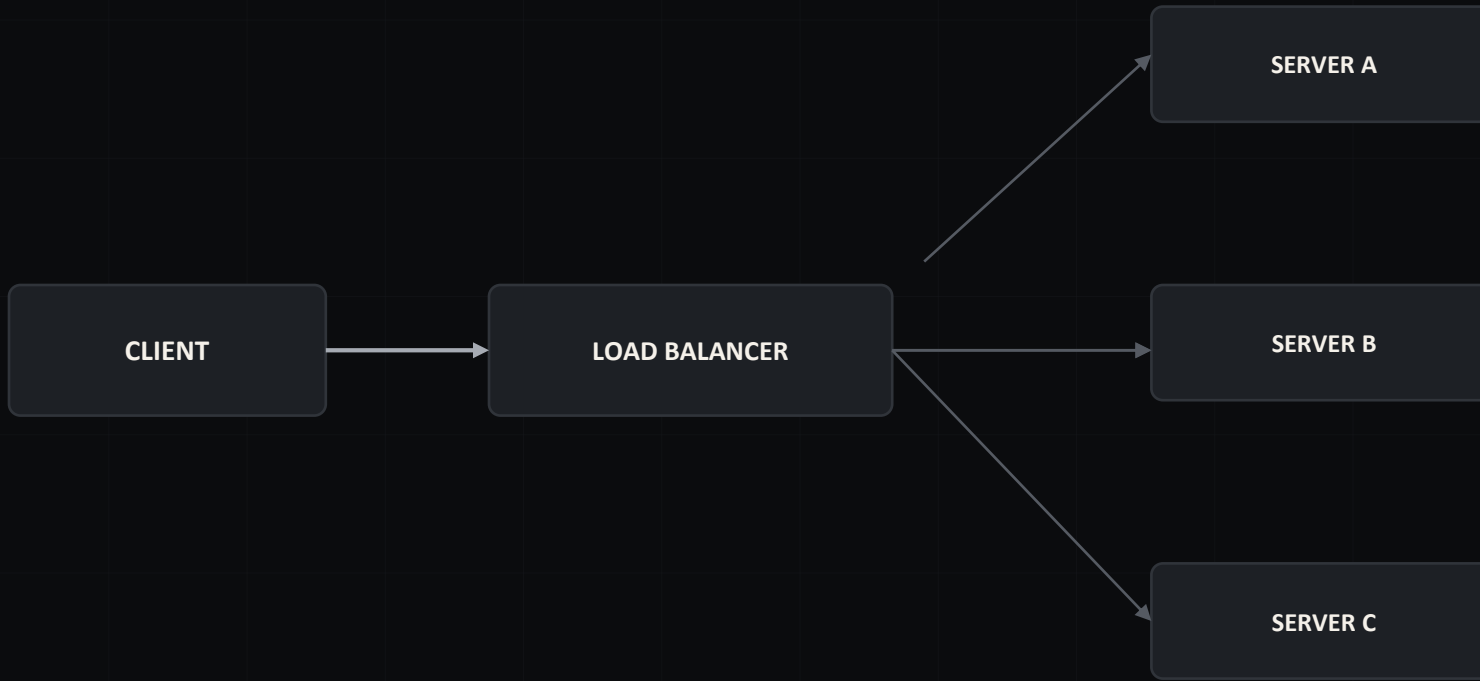
Stronger OAuth/OIDC alignment

05

Extensions + Tasks

Durable work moves into extension model

Stateless at the protocol layer. Stateful when the application needs it.



Explicit application state

incident_id
workflow_id
basket_id

Passed as ordinary tool arguments
when needed.

My MCP checklist today.

1

Is this a tool or a resource?

2

Is the tool narrower than it needs to be?

3

Is authorization enforced server-side?

4

What happens when context is untrusted?

5

Which actions require approval?

6

Can I audit the full tool invocation?

The realization

~~I was building AI agents.~~

**I was building integrations
around an LLM.**

MCP gave that integration work a cleaner boundary and a reusable contract.

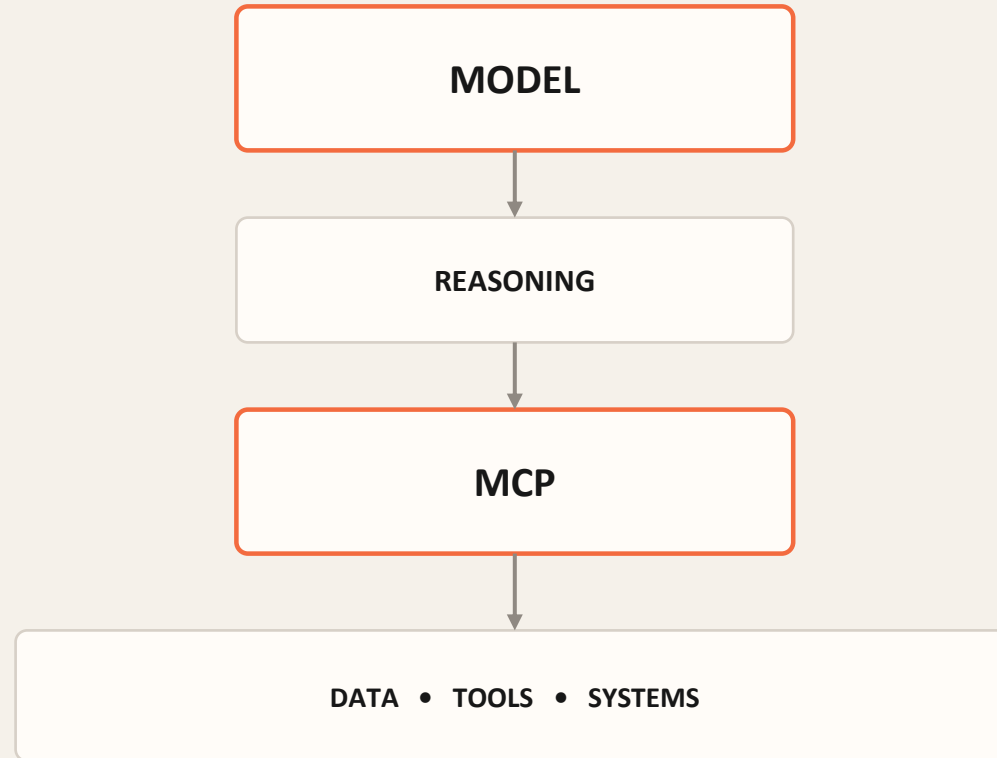
What MCP changed

- 01 Integrations became capabilities.

- 02 Capabilities became reusable.

- 03 Permissions became impossible to ignore.

The model is only one part of the agent.



**MCP didn't make my model smarter.
It changed how I give intelligence capabilities.**

Sources & further reading

1

MCP Architecture overviewmodelcontextprotocol.io/docs/learn/architecture

2

Understanding MCP serversmodelcontextprotocol.io/docs/learn/server-concepts

3

The 2026-07-28 Specificationblog.modelcontextprotocol.io/posts/2026-07-28/

4

Understanding Authorization in MCPmodelcontextprotocol.io/docs/tutorials/security/authorization

5

OWASP MCP Top 10owasp.org/www-project-mcp-top-10/

I Was Building AI Agents the Wrong Way.

Here's What MCP Changed.

UMBRACO
INDIA FESTIVAL 26

